

AWS CloudTrail Threat Analysis (Flaws2 Defender Path)

This project documents my investigation of AWS CloudTrail logs from a simulated cloud security incident. Using AWS CLI, CloudTrail, IAM analysis, and jq, I identified evidence of credential misuse, investigated cross-account role assumptions, and analyzed an Amazon ECR repository misconfiguration.

Objectives

- Download and analyze CloudTrail logs
 - Investigate IAM role activity
 - Identify evidence of credential theft
 - Analyze repository access policies
 - Document security findings and impact
-

Objective 1: Download CloudTrail Logs

Step 1: Setup CLI

Install AWS Vault

Download AWS Vault from:

<https://github.com/99designs/aws-vault>

Verify AWS CLI is Installed and Configured

```
aws --version
```

Configure Credentials

```
aws configure
```

Credentials were configured using temporary lab credentials supplied by the environment.

- Default region name: us-east-1
- Default output format: json

Test Configuration

```
aws sts get-caller-identity
```

Verify that the command returns your IAM identity and account information.

Step 2: Download the Logs

```
aws s3 sync s3://flaws2-logs .
```

You should get a collection of **.json.gz** files within the AWSLogs directory path.

These logs contain CloudTrail records associated with the simulated attack scenario.

This S3 bucket is public so that it can be referenced from Athena later.

Findings

CloudTrail logs were successfully downloaded and contained historical AWS API activity associated with the simulated attack scenario.

Security Impact

CloudTrail serves as a primary source of evidence during AWS incident response investigations and enables analysts to reconstruct attacker activity.

Objective 2

Create a Dedicated Log Directory

```
mkdir C:\Users\<USERNAME>\flaws2logs
```

```
cd C:\Users\<USERNAME>\flaws2logs
aws s3 sync s3://flaws2-logs .
```

```
Mode                LastWriteTime         Length Name
----                -
d-----          6/12/2026   4:28 PM             flaws2logs
download: s3://flaws2-logs/AWSLogs/653711331788/CloudTrail/us-east-1/2018/11/28/653711331788_CloudTrail_us-east-1_20181128T2310Z_A1lhv3sWzRIBFVK.json.gz to AWSLogs\6537113
31788\CloudTrail\us-east-1\2018\11\28\653711331788_CloudTrail_us-east-1_20181128T2310Z_A1lhv3sWzRIBFVK.json.gz
download: s3://flaws2-logs/AWSLogs/653711331788/CloudTrail/us-east-1/2018/11/28/653711331788_CloudTrail_us-east-1_20181128T2310Z_7J9NEIxrjJsrIXsd.json.gz to AWSLogs\6537113
31788\CloudTrail\us-east-1\2018\11\28\653711331788_CloudTrail_us-east-1_20181128T2310Z_7J9NEIxrjJsrIXsd.json.gz
download: s3://flaws2-logs/AWSLogs/653711331788/CloudTrail/us-east-1/2018/11/28/653711331788_CloudTrail_us-east-1_20181128T2305Z_zK1MhON7EpHala9u.json.gz to AWSLogs\6537113
31788\CloudTrail\us-east-1\2018\11\28\653711331788_CloudTrail_us-east-1_20181128T2305Z_zK1MhON7EpHala9u.json.gz
download: s3://flaws2-logs/AWSLogs/653711331788/CloudTrail/us-east-1/2018/11/28/653711331788_CloudTrail_us-east-1_20181128T2310Z_rp9i9zxR2Vcpqfnz.json.gz to AWSLogs\6537113
31788\CloudTrail\us-east-1\2018\11\28\653711331788_CloudTrail_us-east-1_20181128T2310Z_rp9i9zxR2Vcpqfnz.json.gz
download: s3://flaws2-logs/AWSLogs/653711331788/CloudTrail/us-east-1/2018/11/28/653711331788_CloudTrail_us-east-1_20181128T2305Z_83VTWZ8Z0kiEC7Lq.json.gz to AWSLogs\6537113
31788\CloudTrail\us-east-1\2018\11\28\653711331788_CloudTrail_us-east-1_20181128T2305Z_83VTWZ8Z0kiEC7Lq.json.gz
download: s3://flaws2-logs/AWSLogs/653711331788/CloudTrail/us-east-1/2018/11/28/653711331788_CloudTrail_us-east-1_20181128T2310Z_jQajCuioBojD8I4y.json.gz to AWSLogs\6537113
31788\CloudTrail\us-east-1\2018\11\28\653711331788_CloudTrail_us-east-1_20181128T2310Z_jQajCuioBojD8I4y.json.gz
download: s3://flaws2-logs/AWSLogs/653711331788/CloudTrail/us-east-1/2018/11/28/653711331788_CloudTrail_us-east-1_20181128T2310Z_jW5Hfntz7kOnvcP.json.gz to AWSLogs\6537113
31788\CloudTrail\us-east-1\2018\11\28\653711331788_CloudTrail_us-east-1_20181128T2310Z_jW5Hfntz7kOnvcP.json.gz
download: s3://flaws2-logs/AWSLogs/653711331788/CloudTrail/us-east-1/2018/11/28/653711331788_CloudTrail_us-east-1_20181128T2235Z_cR9ra7OH1rytWYXY.json.gz to AWSLogs\6537113
31788\CloudTrail\us-east-1\2018\11\28\653711331788_CloudTrail_us-east-1_20181128T2235Z_cR9ra7OH1rytWYXY.json.gz
```

Verify Logs Exist

```
dir AWSLogs
```

```
Mode                LastWriteTime         Length Name
----                -
d-----          6/12/2026   4:28 PM             653711331788
```

Verify AWS Configuration File Exists

```
notepad $HOME\.aws\config
```

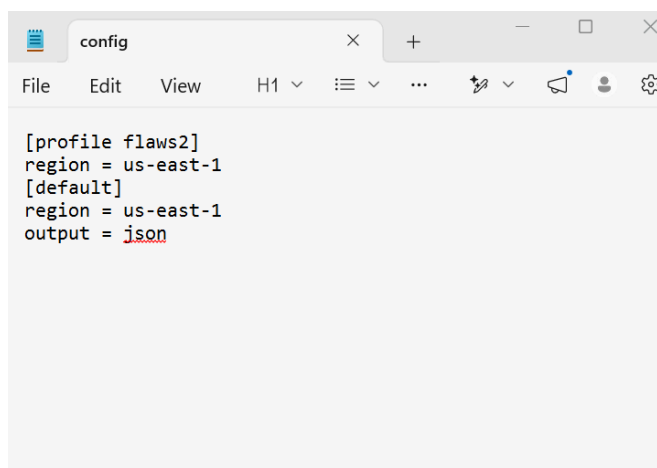
Add a Profile for the Target

```
[profile target_security]
region=us-east-1
output=json
source_profile=default
role_arn=[REDACTED]
```

Save the file.

Test the Profile

```
aws --profile target_security sts get-caller-identity
```



```
config
File Edit View H1  ...  ...
[profile flaws2]
region = us-east-1
[default]
region = us-east-1
output = json
```

Enumerate Accessible S3 Buckets

```
aws --profile target_security s3 ls
```

```
2018-11-20 14:50:08 flaws2.cloud
2018-11-20 13:45:26 level1.flaws2.cloud
2018-11-20 20:41:16 level2-g9785tw8478k4awxtbox9kk3c5ka8iiz.flaws2.cloud
2018-11-26 14:47:22 level3-oc6ou6dnkw8sszwvdraxc5t5udrsw3s.flaws2.cloud
2018-11-27 15:37:27 the-end-962b72bjahfm5b4wcktm8t9z4sapemjb.flaws2.cloud
```

Findings

A cross-account role was configured to access investigation resources within the target environment.

Security Impact

Cross-account roles provide controlled access between AWS accounts and are commonly used by security teams for centralized monitoring and investigation.

Objective 3

Install jq

```
winget install jqlang.jq
```

Re-open PowerShell and verify jq is installed.

```
jq --version
```

Decompress CloudTrail Logs

Gzip is not installed on Windows by default, but PowerShell can unzip .gz files directly.

```
Get-ChildItem -Recurse -Filter *.json.gz | ForEach-Object {
    $infile = $_.FullName
    $outfile = $infile -replace '\.gz$', ''
    $inputStream = [System.IO.File]::OpenRead($infile)
```

```

        $outputStream = [System.IO.File]::Create($outfile)
        $gzipStream = New-Object
        System.IO.Compression.GzipStream($inputStream,
        [System.IO.Compression.CompressionMode]::Decompress)
        $gzipStream.CopyTo($outputStream)
        $gzipStream.Close()
        $outputStream.Close()
        $inputStream.Close()
    }

```

Verify JSON Files Exist

```
Get-ChildItem -Recurse -Filter *.json | Select-Object -First 5
```

```

Mode                LastWriteTime         Length Name
-----
-a----          6/12/2026   5:29 PM             3322 653711331788_
                        CloudTrail_us
                        -east-1_20181
                        128T2235Z_cR9
                        ra70HlrytWyXY
                        .json
-a----          6/12/2026   5:29 PM             1217 653711331788_
                        CloudTrail_us
                        -east-1_20181
                        128T2305Z_83V

```

Run jq Test

```

Get-ChildItem -Recurse -Filter *.json | ForEach-Object {
    Get-Content $_.FullName
} | jq '.'

```

```

{
  "Records": [
    {
      "eventVersion": "1.05",
      "userIdentity": {
        "type": "AWSService",
        "invokedBy": "ecs-tasks.amazonaws.com"
      },
      "eventTime": "2018-11-28T22:31:59Z",
      "eventSource": "sts.amazonaws.com",
      "eventName": "AssumeRole",
      "awsRegion": "us-east-1",
      "sourceIPAddress": "ecs-tasks.amazonaws.com",
      "userAgent": "ecs-tasks.amazonaws.com",
      "requestParameters": {
        "roleSessionName": "d190d14a-2404-45d6-9113-4eda22d7f2c7",
        "roleArn": "arn:aws:iam::653711331788:role/Level3"
      }
    }
  ]
}

```

Show Only Event Names

```

Get-ChildItem -Recurse -Filter *.json | ForEach-Object {
    Get-Content $_.FullName
} | jq '.Records[] | .eventName'

```



```

2018-11-28T22:31:59Z AssumeRole
2018-11-28T22:31:59Z AssumeRole
2018-11-28T23:03:12Z CreateLogStream
2018-11-28T23:02:56Z GetObject
2018-11-28T23:03:08Z GetObject
2018-11-28T23:03:11Z GetObject
2018-11-28T23:03:20Z Invoke
2018-11-28T23:03:50Z CreateLogStream
2018-11-28T23:03:12Z AssumeRole
2018-11-28T23:03:20Z CreateLogStream
2018-11-28T23:03:13Z CreateLogStream
2018-11-28T23:05:53Z ListImages
2018-11-28T23:03:35Z CreateLogStream
2018-11-28T23:06:17Z BatchGetImage
2018-11-28T23:06:33Z GetDownloadUrlForLayer
2018-11-28T23:02:56Z GetObject
2018-11-28T23:02:56Z GetObject
2018-11-28T23:02:56Z GetObject
2018-11-28T23:02:57Z GetObject
2018-11-28T23:03:08Z GetObject
2018-11-28T23:03:08Z GetObject
2018-11-28T23:03:08Z GetObject
2018-11-28T23:03:08Z GetObject
2018-11-28T23:03:11Z GetObject
2018-11-28T23:03:13Z Invoke
2018-11-28T23:03:14Z GetObject
2018-11-28T23:03:17Z GetObject
2018-11-28T23:03:18Z GetObject
2018-11-28T23:04:54Z ListObjects
2018-11-28T23:05:10Z GetObject
2018-11-28T23:05:12Z GetObject
2018-11-28T23:05:12Z GetObject
2018-11-28T23:07:08Z GetObject
2018-11-28T23:07:08Z GetObject
2018-11-28T23:09:28Z ListBuckets
2018-11-28T23:09:36Z GetObject

```

Show Useful Investigation Fields

```

Get-ChildItem -Recurse -Filter *.json | ForEach-Object {
    Get-Content $_.FullName
} | jq -r '.Records[] | [.eventTime, .sourceIPAddress,
.userIdentity.arn, .userIdentity.accountId, .userIdentity.type,
.eventName] | @tsv'

```

Findings

Using jq allowed CloudTrail records to be filtered into a more readable format. By extracting timestamps, source IP addresses, IAM identities, and event names, it became easier to reconstruct user activity and identify potentially suspicious behavior within the environment.

Security Impact

CloudTrail analysis provides visibility into authentication events, resource access, privilege changes, and other actions that are critical during incident response investigations.

Objective 4: Identify Credential Theft

Purpose

The goal was to determine whether an IAM role was being assumed by an unauthorized principal.

Investigation

I reviewed the trust policy associated with the level3 IAM role using:

```
aws --profile target_security iam get-role --role-name level3
```

```
{
  "Role": {
    "Path": "/",
    "RoleName": "level3",
    "RoleId": "AROAJQMBDNUMIKLZKMF64",
    "Arn": "arn:aws:iam::653711331788:role/level3",
    "CreateDate": "2018-11-23T17:55:27+00:00",
    "AssumeRolePolicyDocument": {
      "Version": "2012-10-17",
      "Statement": [
        {
          "Sid": "",
          "Effect": "Allow",
          "Principal": {
            "Service": "ecs-tasks.amazonaws.com"
          },
          "Action": "sts:AssumeRole"
        }
      ]
    },
    "Description": "Allows ECS tasks to call AWS services on your behalf.",
    "MaxSessionDuration": 3600,
    "RoleLastUsed": {
      "LastUsedDate": "2026-05-19T09:10:43+00:00",
      "Region": "us-east-1"
    }
  }
}
```

I then compared the trusted principal defined in the policy against the principals observed within CloudTrail logs.

Findings

CloudTrail activity indicated role assumptions that did not align with the intended trust relationship.

Security Impact

Unauthorized role assumptions can allow attackers to gain access to AWS resources and move laterally within the environment.

Objective 5: Identify a Publicly Accessible Resource

Investigation

I reviewed the Amazon ECR repository policy:

```
aws --profile target_security ecr get-repository-policy
--repository-name level2
```

Findings

The repository policy contained:

```
"Principal": "*"
```

which granted access to any principal.

```
{
  "registryId": "653711331788",
  "repositoryName": "level2",
  "policyText": "{\n  \"Version\" : \"2008-10-17\",\n  \"Statement\" : [ {\n    \"Sid\" : \"AccessControl\",\n    \"Effect\" : \"Allow\",\n    \"Principal\" : \"*\",\n    \"Action\" : [ \"ecr:GetDownloadUrlForLayer\", \"ecr:BatchGetImage\", \"ecr:BatchCheckLayerAvailability\", \"ecr:ListImages\", \"ecr:DescribeImages\" ]\n  } ]\n}"
```

Security Impact

This configuration exposed repository information to unauthenticated users and allowed attackers to enumerate container image metadata. Publicly accessible repositories can leak information about internal applications and infrastructure.

Key Findings

1. CloudTrail logs revealed suspicious IAM activity consistent with credential misuse.
2. Cross-account role assumptions were used to access resources.
3. An Amazon ECR repository was publicly accessible due to an overly permissive repository policy.
4. CloudTrail and jq were effective tools for reconstructing attacker activity and identifying misconfigurations.

Skills Demonstrated

- AWS CloudTrail Analysis
- AWS IAM
- AWS CLI
- Incident Response
- Threat Hunting

- Log Analysis
- Cloud Security
- jq
- PowerShell